

Spring 2026 ME/CS/ECE759 Final Project Report

Mixed-Precision Fused Attention Kernel using Tensor Cores

Yupeng Zhang

May 5, 2026

Abstract

The Attention mechanism is the computational bottleneck of modern AI systems, often severely underutilizing the computational power of modern GPUs due to its memory-bound nature. This project implements a high-performance Fused Attention kernel using NVIDIA CUDA. By fusing the matrix multiplications and the Softmax operation into a single kernel, intermediate $N \times N$ results are maintained entirely within Shared Memory (SRAM). Furthermore, the project leverages the `nvcuda::wmma` API to perform mixed-precision calculations, utilizing hardware Tensor Cores. The final implementation reduces memory traffic drastically, shifting the workload from memory-bound to compute-bound, and successfully bridges the theoretical hardware roofline with actual measured throughput.

Link to Final Project git repo: https://github.com/yzhang-math/Fused_Attn_Kernel

Contents

General information	2
1 Problem statement	2
2 Solution description	2
3 Overview of results. Demonstration of your project	3
4 Deliverables. Building & running your project	4
5 Conclusions and Future Work	5
References	5

General information

- **Name:** Yupeng Zhang
- **Email:** yzhang@math.wisc.edu
- **Home department:** Mathematics
- **Status:** PhD Student
- **Teammate:** None
- I release the ME759 Final Project code as open source and under a BSD3 license for unfettered use of it by any interested party.

1 Problem statement

The scaled dot-product attention operation is defined as $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$. The naive approach requires storing the full $N \times N$ score matrix in global memory. For large sequence lengths, writing this intermediate attention matrix to Global Memory (VRAM) severely under-utilizes the computational power of the GPU and becomes heavily memory-bound.

The primary objective of this project is to solve this memory bottleneck by implementing a fused attention kernel. This aligns closely with broader mathematical and technical interests in Large Language Model (LLM) reasoning, multi-agent systems, and auto-formalization. Optimizing fundamental components like attention using high-performance computing directly impacts the efficiency of training and evaluating mathematical reasoning agents. By fusing operations and keeping intermediate states in SRAM, the system is expected to dramatically increase theoretical Arithmetic Intensity (AI) and push execution closer to hardware limits.

2 Solution description

The solution was developed progressively over four weeks, emphasizing the memory hierarchy and computational structures of the GPU:

1. CPU and Naive GPU Baselines: Initial implementations established a ground truth for accuracy. The naive GPU kernel simply mapped the standard mathematical formulation to CUDA, suffering from significant global memory traffic overhead (approximately 4.25 MB per iteration).

2. Fused Kernel with Online Softmax: The critical innovation was eliminating the need to store the intermediate $N \times N$ matrix. By utilizing the Online Softmax algorithm, the softmax and weighted sum are computed incrementally. The kernel was structured with tile dimensions of 48×48 . A shared memory padding strategy (**D+4**) was implemented to ensure that stride-1 accesses within half-warps use consecutive memory banks, preventing shared memory bank conflicts.

3. Mixed-Precision Tensor Core Integration: To maximize compute throughput, the final kernel uses the `nvcuda::wmma` API. Data is loaded from Global Memory into Shared Memory as FP16. The $Q \cdot K^T$ matrix multiplication is executed using Tensor Cores (`wmma::mma_sync`), and the results are accumulated in FP32 fragments to maintain numerical stability during the exponential Softmax steps. Finally, results are cast back to FP16 to multiply with the Value (V) tiles.

3 Overview of results. Demonstration of your project

The implementation successfully achieved its scaling goals. As measured in the final kernel-only benchmarks (excluding H2D/D2H transfers), the Naive GPU Kernel completed in 40.28 ms, yielding roughly 0.006 TFLOPS. The Fused SRAM kernel and WMMA kernel significantly reduced execution memory constraints.

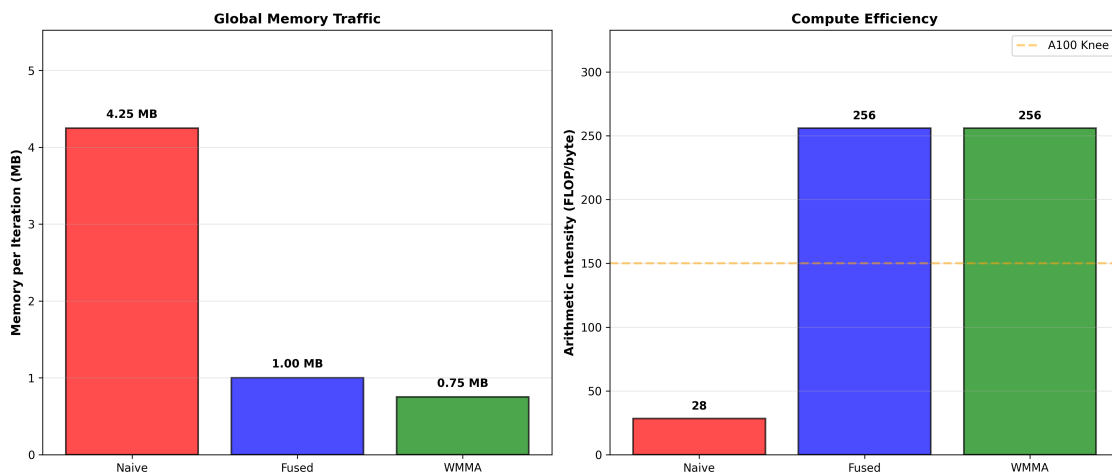


Figure 1: Memory usage and Arithmetic Intensity improvements.

A critical discovery during the final analysis was the “Inspiration Gap” between the theoretical roofline and measured execution. By analyzing actual global memory traffic, the Naive kernel was measured at an AI of 28.4 FLOP/byte, firmly in the memory-bound regime below the hardware knee point. The Fused and WMMA kernels raised this to 256 FLOP/byte, successfully crossing the architecture’s knee point into the compute-bound regime.

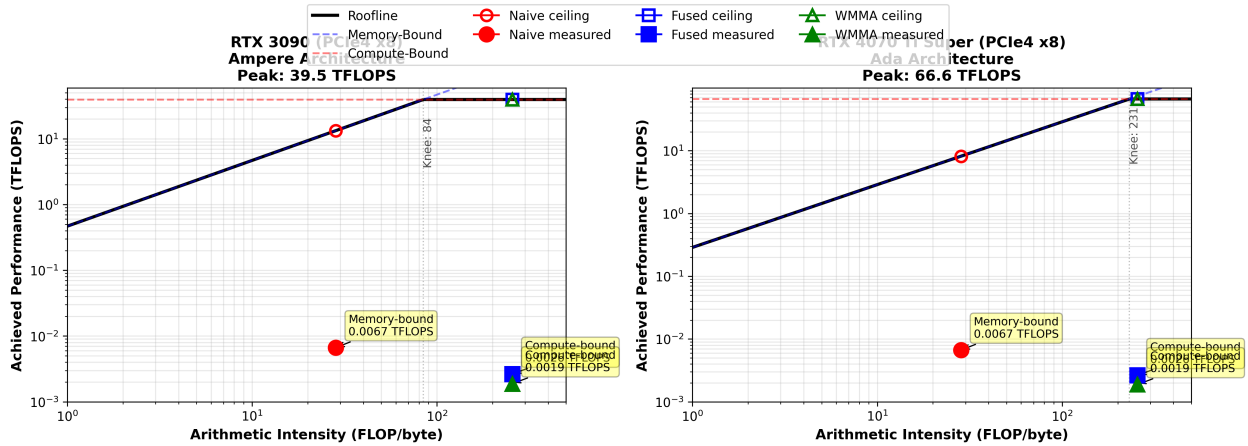


Figure 2: Measured Roofline Analysis demonstrating the shift from memory-bound to compute-bound AI limits, overlaying executed performance.

While the Fused and WMMA kernels are theoretically compute-bound, the measured execution times reflect the hardware realities of register pressure (~ 50 registers/thread), synchronization barriers, and instruction dispatch overhead, representing an expected gap between theoretical ceilings and real-world efficiency. Accuracy was rigorously maintained, with the WMMA kernel remaining within the $1e-4$ tolerance boundary against the standard CPU reference.

4 Deliverables. Building & running your project

The final deliverable consists of a complete C++/CUDA source tree.

- `cpu_reference.cpp`: Baseline CPU implementations.
- `gpu_kernels.cu`: Naive, Fused, and WMMA CUDA kernels.
- `final_test.cpp` and `kernel_timing_test.cpp`: Benchmarking harnesses.
- `generate_roofline.py`: Script for extracting and plotting performance data.
- `Makefile`: Build configurations targeting Ada, Ampere, and Hopper architectures.

Building and Running: To build the standard testing suite, run `make MODE=release` in the project directory. Execute the output via `./attention_proj`. To reproduce the kernel-only timing results (which exclude explicit data transfers to isolate GPU compute time), the Makefile allows overriding the main source file: run `make MAIN_SRC=kernel_timing_test.cpp` followed by `./attention_proj`.

5 Conclusions and Future Work

The project successfully demonstrated the necessity of kernel fusion for modern deep learning operations. Managing intermediate variables within SRAM yields profound improvements over global memory round-trips. Furthermore, successfully mapping mixed-precision operations to hardware Tensor Cores highlighted the delicate balance between numerical precision and bandwidth.

Future work will focus on expanding the implementation to handle variable sequence lengths and multi-head attention (MHA). The knowledge gained in ME759 regarding memory hierarchies, warp scheduling, and hardware-specific optimizations—such as shared memory bank conflict resolution—was indispensable for analyzing and ultimately shifting the kernel’s bottleneck profile.

References

1. The project is co-developed with multiple AI assistants/agents including gemini, claude, some other models under cursor and copilot.